

Pipelined AHB-to-APB Bridge

RTL Design, Verification and Synthesis

Agenda

- Architecture
- Block Diagram
- AHB Slave Interface (Role, RTL, Waveform)
- APB Controller (Role, FSM, RTL, Waveforms)
- Top-Level Integration
- Verification (AHB Master BFM, Testbench)
- Synthesis
- Conclusion

Architecture

Overview

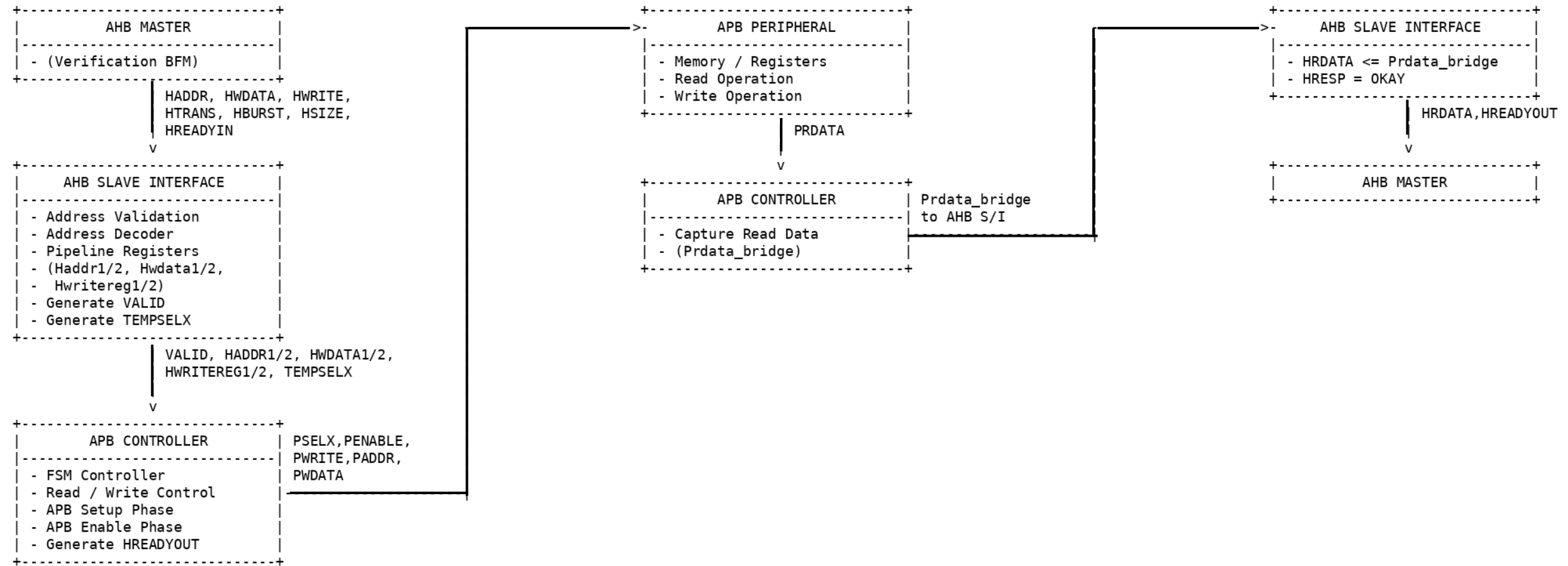
The proposed AHB–APB Bridge provides communication between a high-speed AHB bus and low-speed APB peripherals. The bridge consists of an **AHB Master (verification model)**, an **AHB Slave Interface**, an **APB Controller**, and an **APB Peripheral**. The AHB Slave Interface captures and pipelines AHB transactions, while the APB Controller converts them into APB-compliant read and write operations using a finite state machine (FSM). The APB Peripheral performs the requested operation and returns read data to the AHB Master through the bridge.

Key Functional Blocks

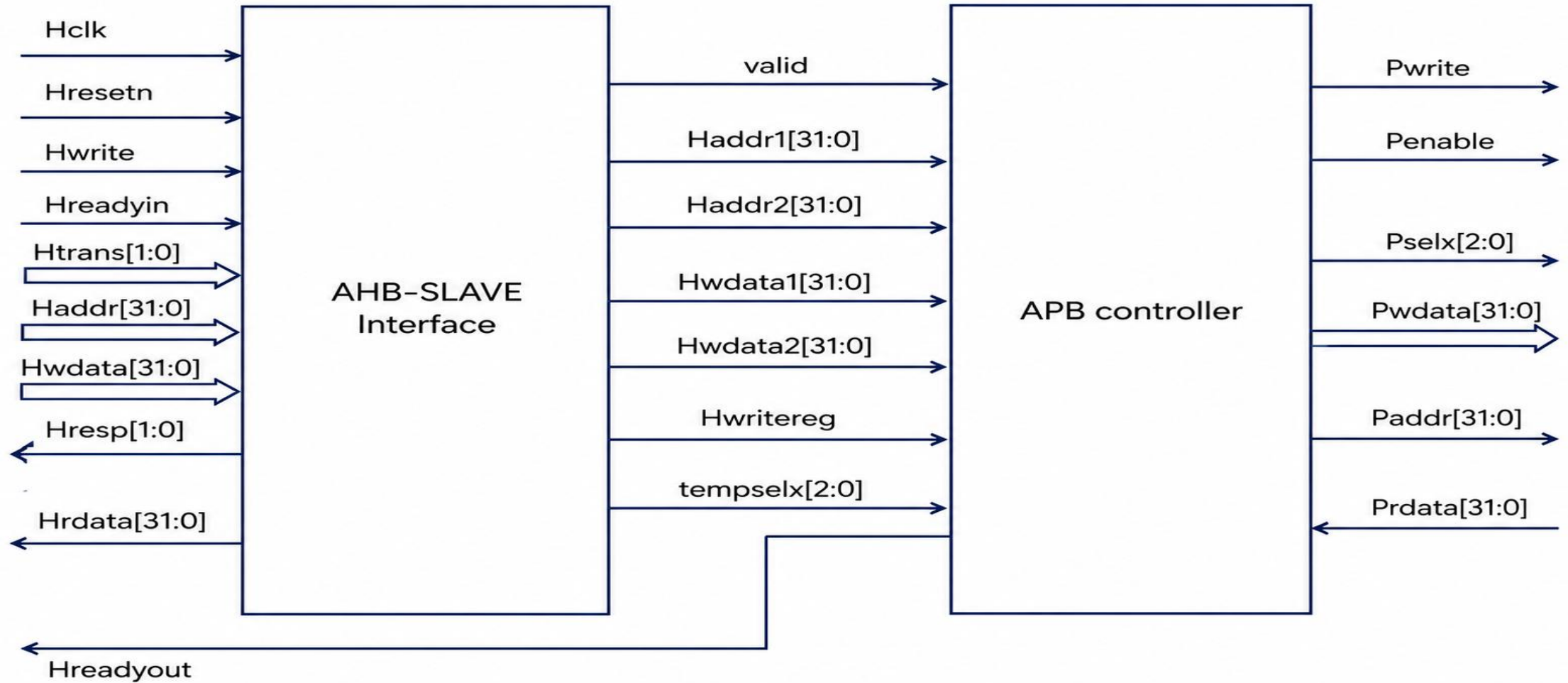
- **AHB Master** – Generates configurable single and burst read/write transactions.
- **AHB Slave Interface** – Validates AHB transfers, decodes addresses, and pipelines address, data, and control signals.
- **APB Controller (FSM)** – Converts AHB transactions into APB Setup and Enable phases.
- **APB Peripheral** – Executes read/write operations and returns read data.
- **Read Data Path** – Transfers PRDATA back to the AHB Master through the bridge.
- **Control Signals** – HREADYOUT and HRESP synchronize and acknowledge completed transfers.

Architecture

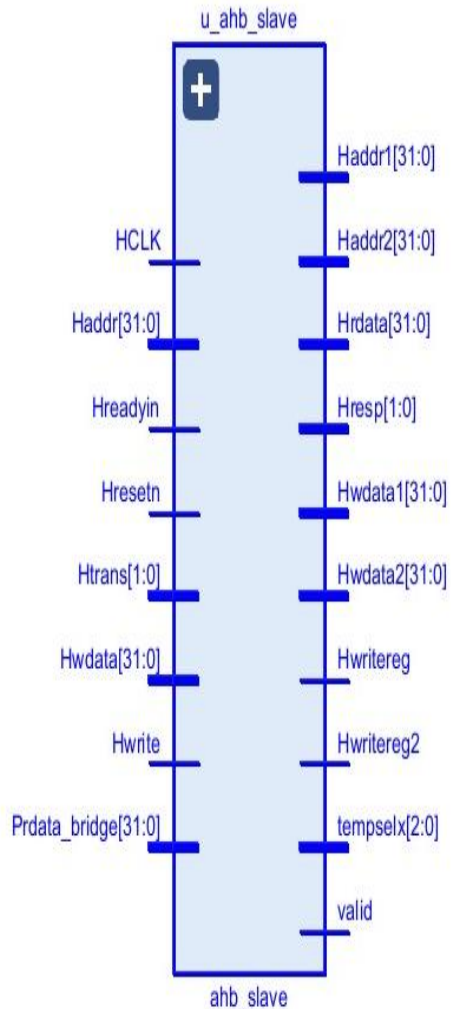
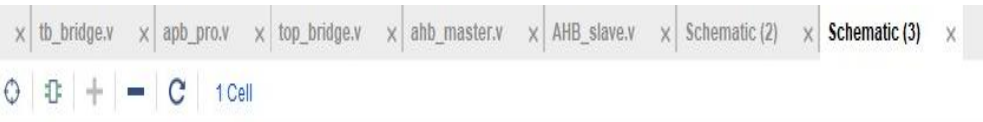
Figure: Overall Architecture of the AHB–APB Bridge



Block Diagram



AHB Slave Interface - Role and Ports



Purpose

- Interfaces the AHB Master with the APB Controller.
- Receives AHB address, control, and write data signals.
- Performs address decoding and generates peripheral select signals.
- Pipelines AHB address, data, and control information for APB transactions.
- Returns read data received from the APB Controller back to the AHB Master.

Functionality

Address Validation

- Checks whether the incoming address belongs to the APB address space.
- Verifies that the transfer type is **NONSEQ** or **SEQ**.
- Generates the **valid** signal for the APB Controller.

Address Decoding

- Decodes the AHB address into the corresponding APB peripheral.
- Generates the **tempselx** signal.

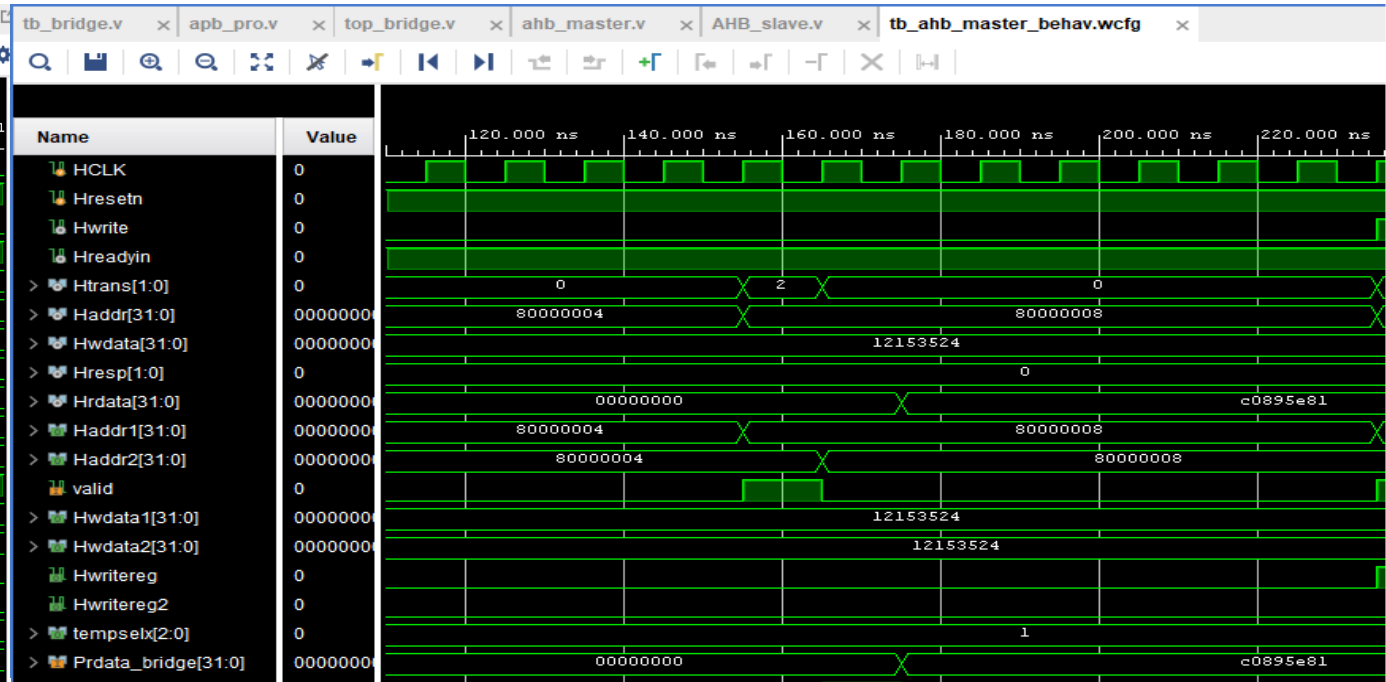
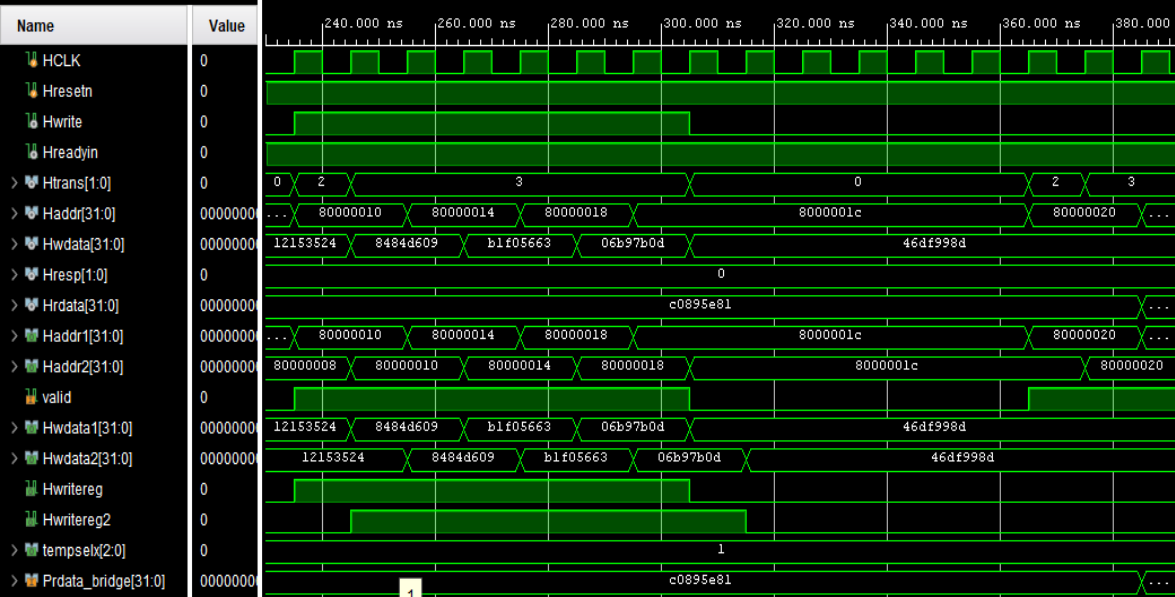
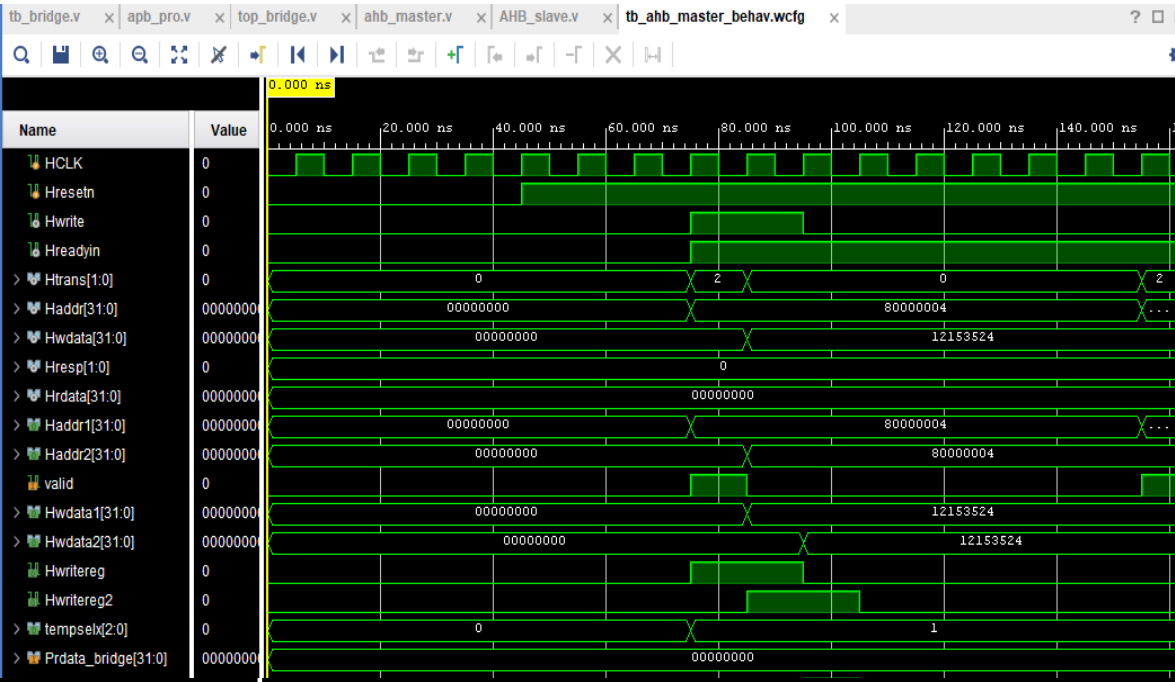
Pipeline Registers

- Stores address in **Haddr1** and **Haddr2**.
- Stores write data in **Hwdata1** and **Hwdata2**.
- Stores write/read control in **Hwritereg** and **Hwritereg2**.
- Synchronizes AHB pipelined transfers with the APB protocol.

Read Data Path

- Receives **Prdata_bridge** from the APB Controller.
- Drives the AHB read data output **Hrdata**.

AHB Slave Interface - Waveform



1) SINGLE WRITE

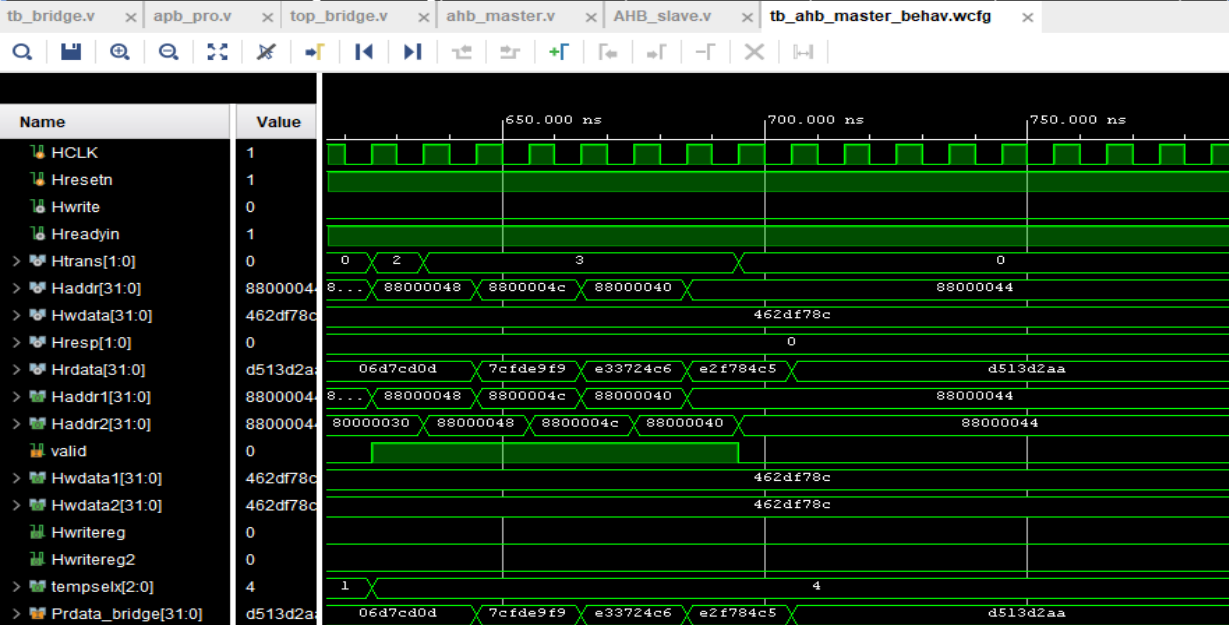
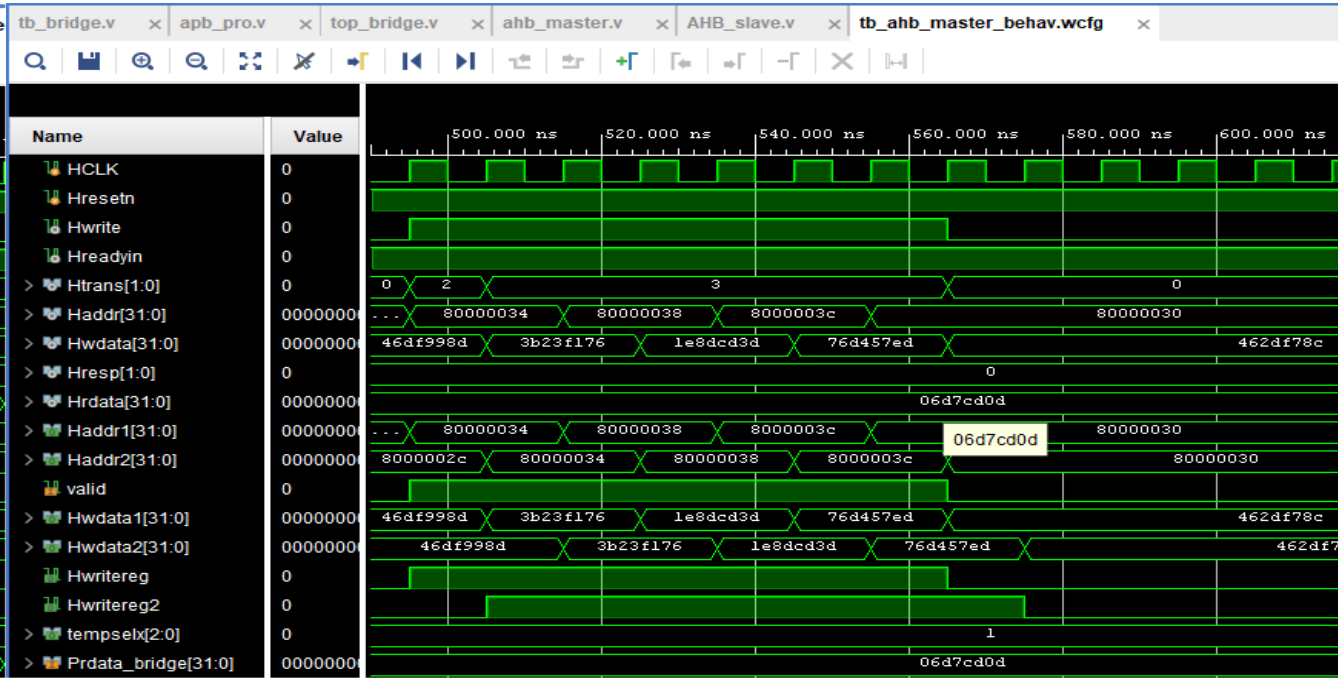
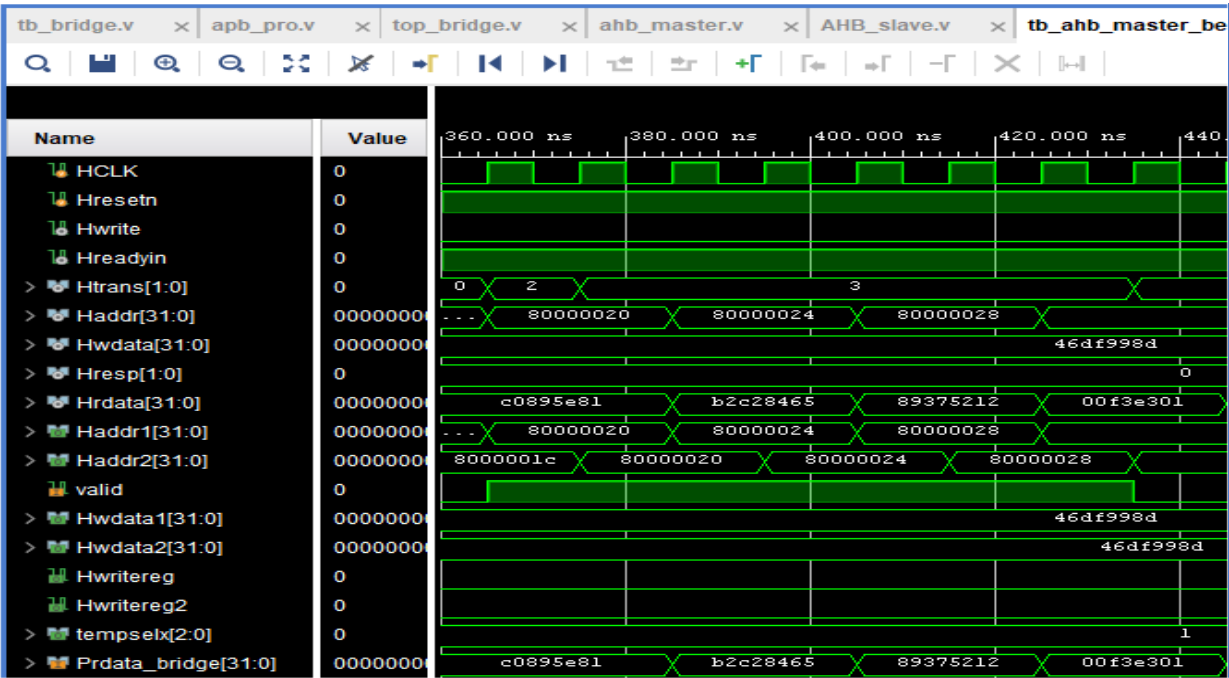
2) SINGLE READ

3) BURST WRITE-INCR4

Inputs: Haddr, Hwdata, Hwrite, Htrans, Hreadyin, Hclk, Hresetn

Outputs: Haddr1-2, Hwdata1-2, Hwritereg-2, tempselx, valid, Hrdata

AHB Slave Interface - Waveform



1) BURST READ – INCR4

2) BURST WRITE – WRAP4

3) BURST READ – WRAP4

Inputs: Haddr, Hwdata, Hwrite, Htrans, Hreadyin, Hclk, Hresetn
Outputs: Haddr1-2, Hwdata1-2, Hwritereg-2, tempselx, valid, Hrdata

APB Controller - Role and Ports

Purpose

- Converts valid AHB transactions into APB-compliant transfers.
- Controls APB read and write operations using a Finite State Machine (FSM).
- Generates APB control signals and synchronizes AHB and APB protocols.
- Controls the APB Setup and Enable phases.
- Returns the transaction completion status to the AHB Master through HREADYOUT.

Functionality

Finite State Machine (FSM)

- Controls the complete APB transfer sequence.
- Supports both read and write transactions.
- Transitions between Idle, Setup, and Enable states based on AHB control signals.

Write Operation

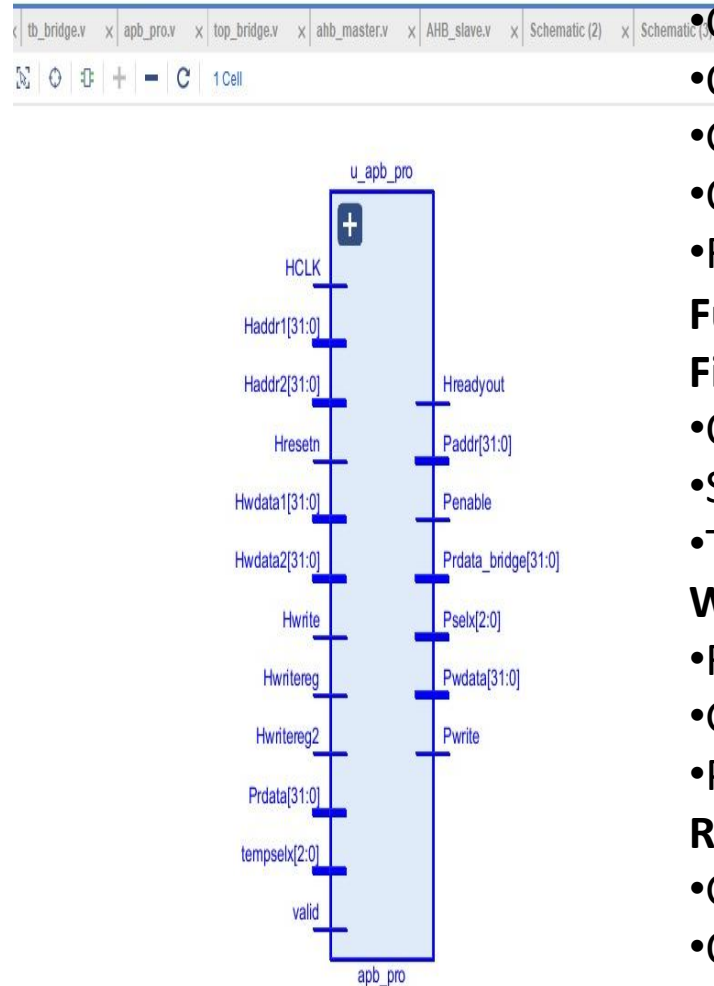
- Receives pipelined address, data, and control signals from the AHB Slave Interface.
- Generates PADDR, PWDATA, PWRITE, and PSELx.
- Performs APB Setup and Enable phases for write transactions.

Read Operation

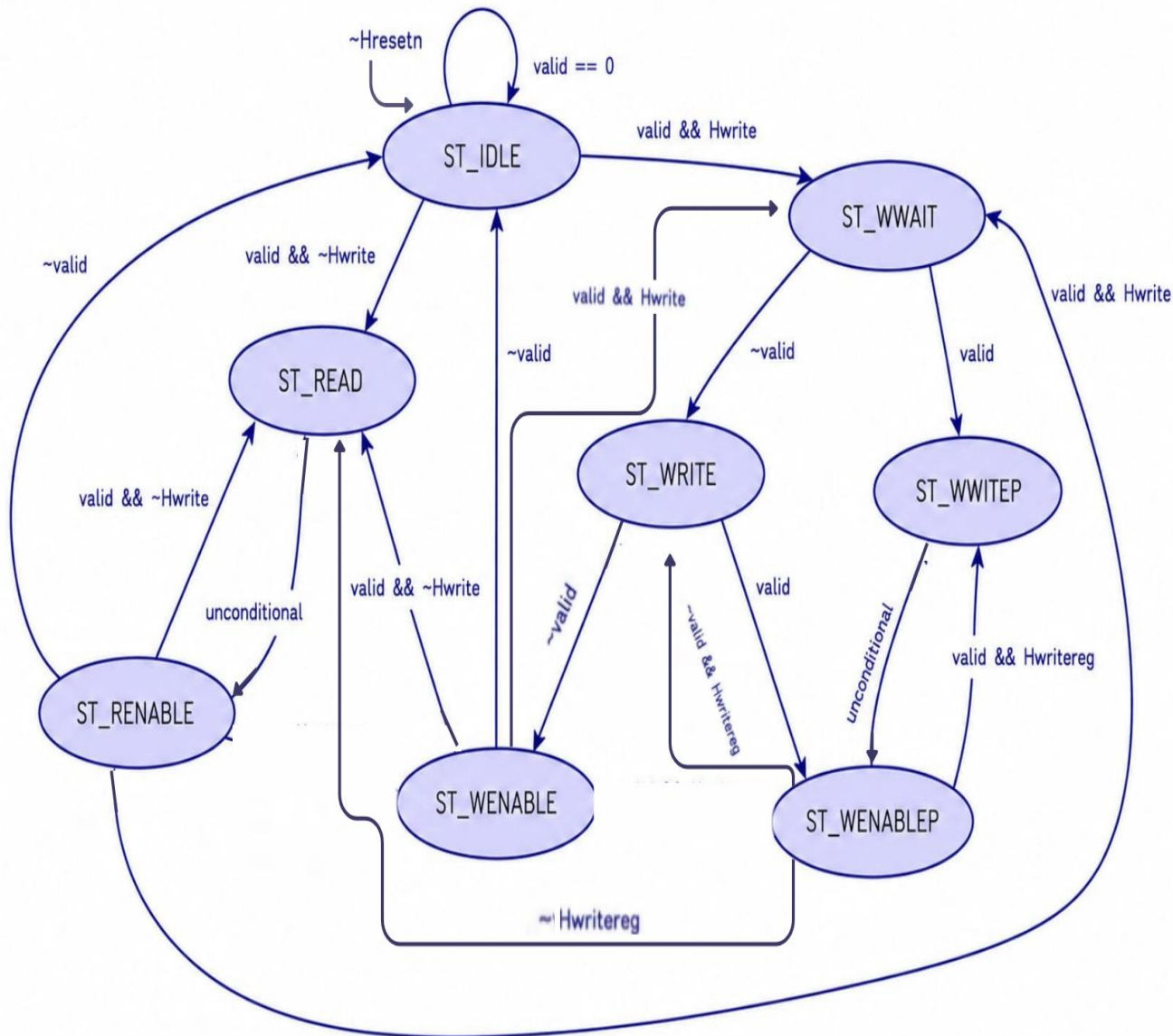
- Generates APB read control signals.
- Captures PRDATA from the APB Peripheral.
- Transfers the read data back to the AHB Slave through Prdata_bridge.

Transfer Synchronization

- Generates HREADYOUT to indicate transaction completion.
- Ensures proper synchronization between the AHB pipeline and APB protocol.



APB Controller - FSM State Diagram



FSM Overview

- Implements the APB protocol using **8 finite states**.
- Controls both **Read** and **Write** transactions.
- Generates APB **Setup** and **Enable** phases.
- Synchronizes AHB pipelined transfers with APB timing.

State Description

State

IDLE

WWAIT

WRITE / WRITEP

WENABLE / WENABLEP

READ

RENABLE

Function

Waits for a valid AHB transaction

Waits for write data from AHB

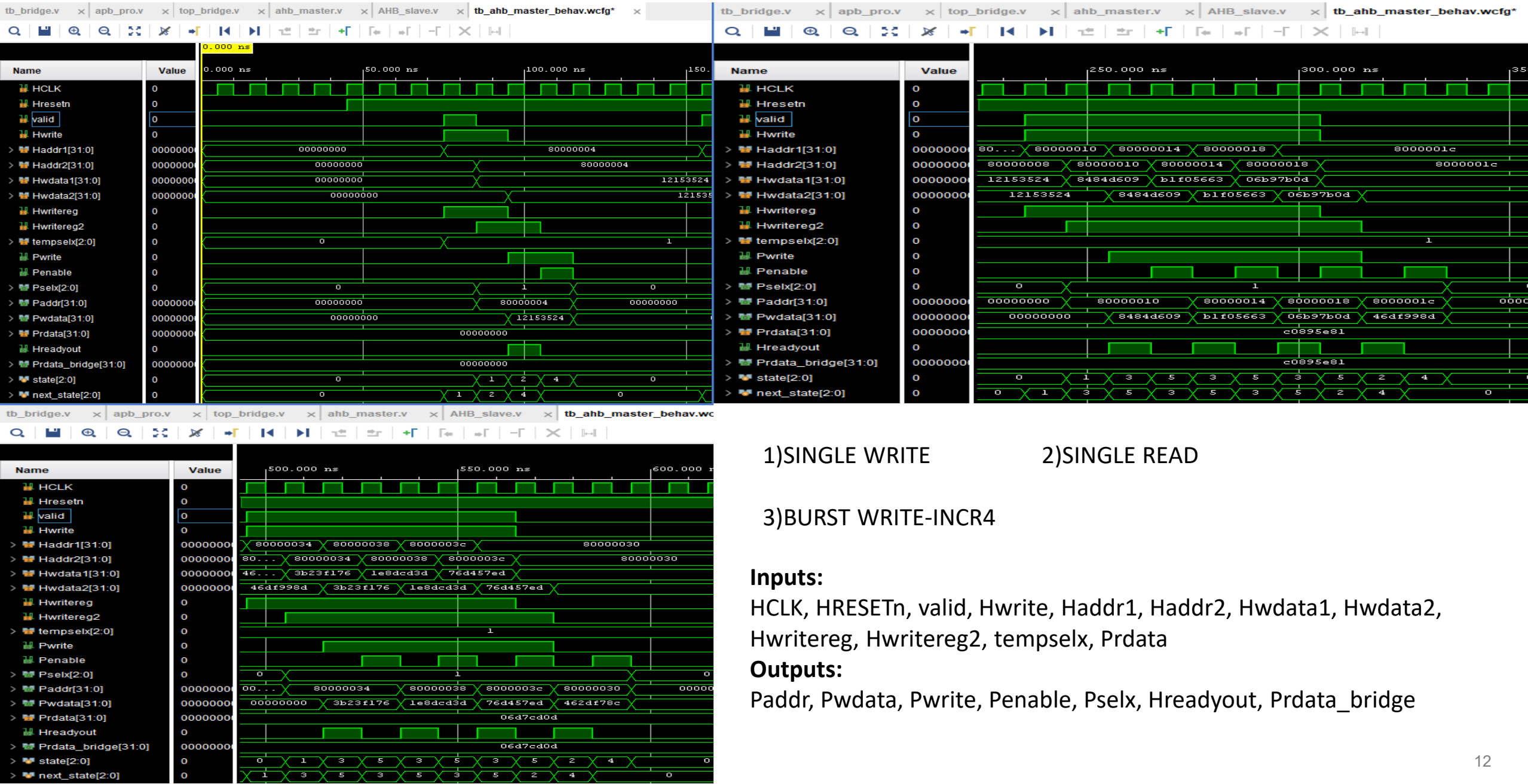
APB Write Setup phase

APB Write Enable phase

APB Read Setup phase

APB Read Enable phase

APB Controller - Write Transaction Waveform



1) SINGLE WRITE

2) SINGLE READ

3) BURST WRITE-INCR4

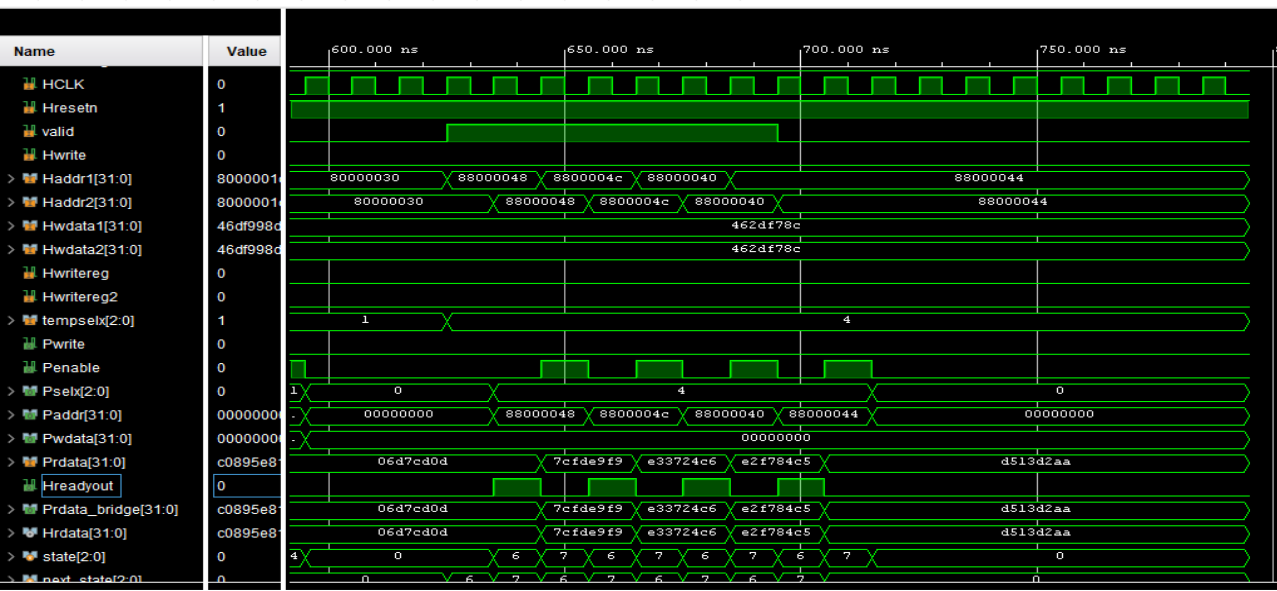
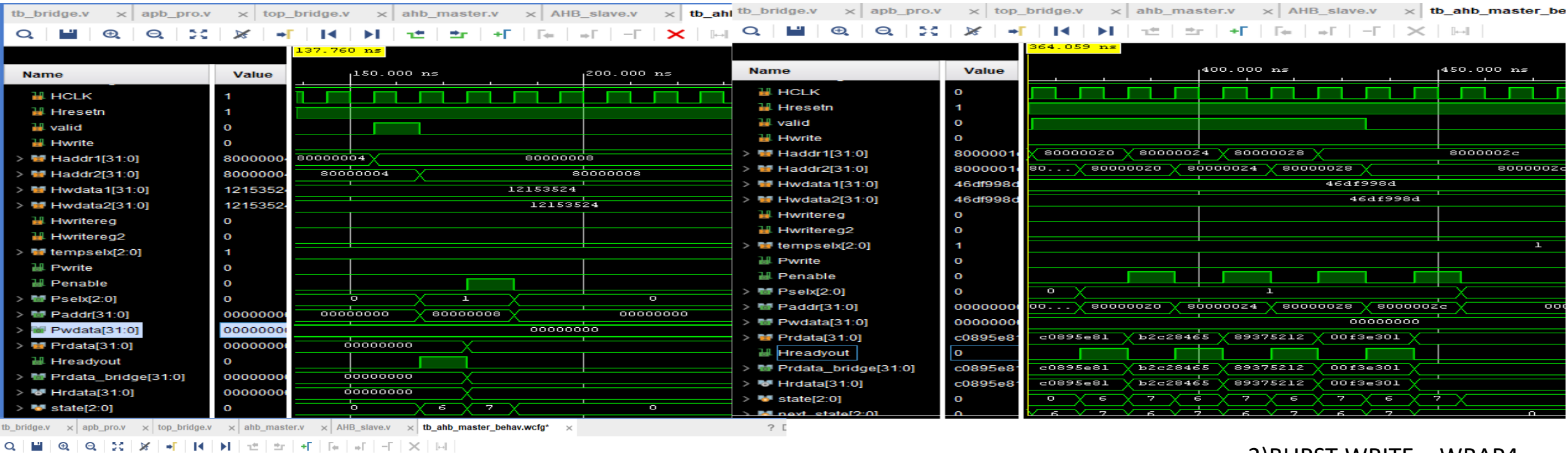
Inputs:

HCLK, HRESETn, valid, Hwrite, Haddr1, Haddr2, Hwdata1, Hwdata2, Hwritereg, Hwritereg2, tempselex, Prdata

Outputs:

Paddr, Pwdata, Pwrite, Penable, Pselx, Hreadyout, Prdata_bridge

APB Controller - Read Transaction Waveform



1)BURST READ – INCR4

2)BURST WRITE – WRAP4

3)BURST READ – WRAP4

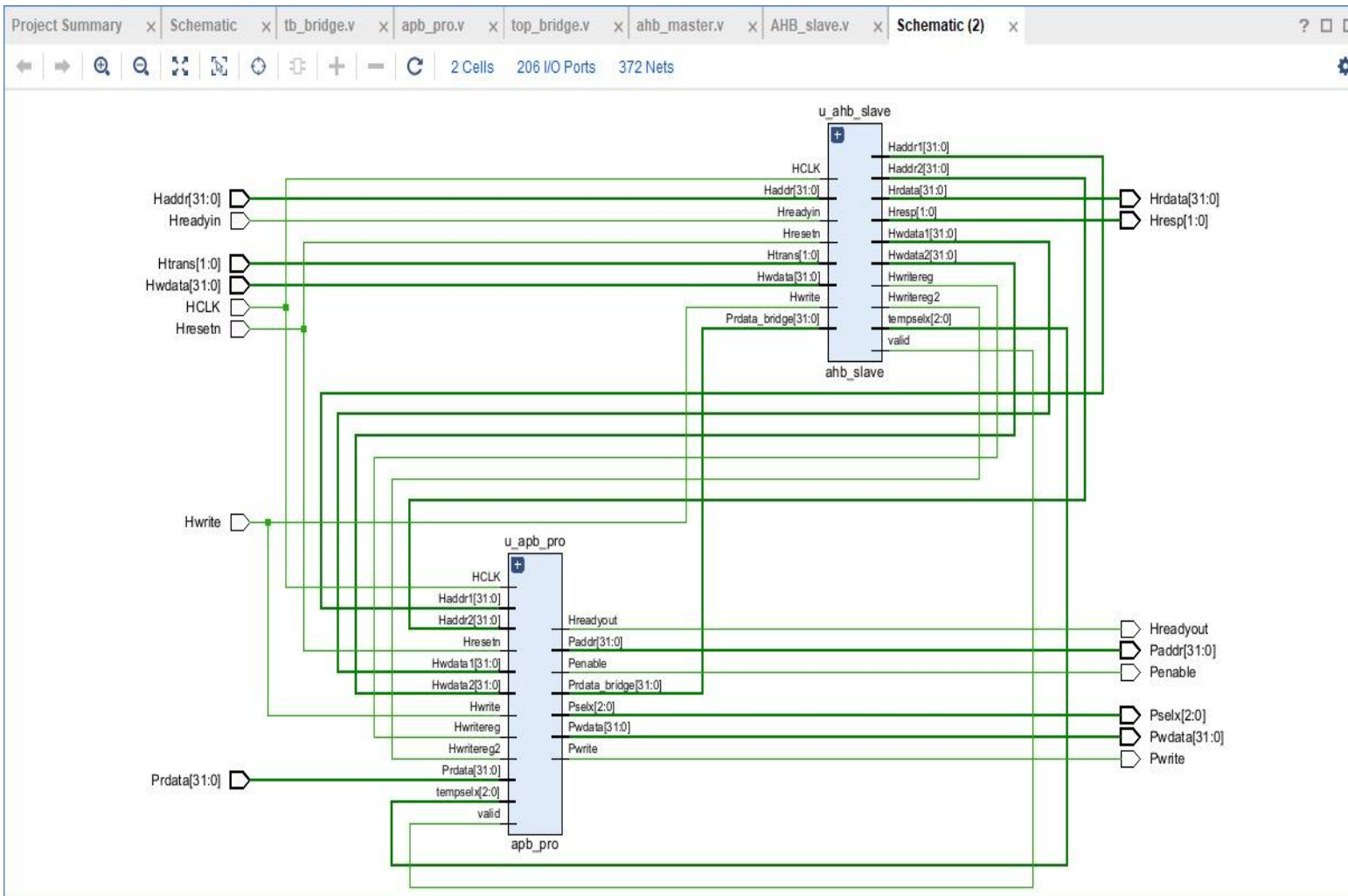
Inputs:

HCLK, HRESETn, valid, Hwrite, Haddr1, Haddr2, Hwdata1, Hwdata2, Hwritereg, Hwritereg2, tempselex, Prdata

Outputs:

Paddr, Pwdata, Pwrite, Penable, Pselx, Hreadyout, Prdata_bridge

Top-Level Wrapper and Elaborated Schematic



top_ahb_apb_bridge

Wires `ahb_slave` directly into `apb_pro`: valid, `Haddr1/2`, `Hwdata1/2`, `Hwritereg/2`, `tempselex` flow from the AHB side; `Prdata_bridge` flows back for the read path.

Top-Level Wrapper

- Instantiates the **AHB Slave Interface** and **APB Controller**.
- Connects all intermediate pipeline signals between the two modules.
- Routes AHB master inputs to the bridge.
- Routes APB control signals to the peripheral.
- Returns APB read data back to the AHB side through `Prdata_bridge`.
- Handles synchronization using the common `HCLK` and `HRESETn`.

Verification - AHB Master BFM

Purpose

- Simulates an AHB Master to verify the functionality of the AHB–APB Bridge.
- Generates configurable AHB read and write transactions.
- Supports both single and burst transfers with different transfer sizes.

Implemented Tasks

1. `ahb_burst_and_single_write()`

- Generates Single and Burst Write transactions.
- Supports SINGLE, INCR, INCR4/8/16, and WRAP4/8/16 bursts.

2. `ahb_burst_and_single_read()`

- Generates Single and Burst Read transactions.
- Stores received data into an internal read buffer for verification.

Features

- Supports Single Read/Write operations.
- Supports Incrementing and Wrapping bursts.
- Automatically generates sequential or wrapped addresses.
- Drives the correct HTRANS sequence (NONSEQ → SEQ → IDLE).
- Buffers read data for result checking.

Task Inputs

Parameter	Description
addr	Starting address of the transaction
Hsize	Transfer size (Byte / Halfword / Word)
Hburst	Burst type (SINGLE, INCR, WRAP4, INCR4, etc.)

Verification - Testbench Stimulus Sequence

Testbench (TB)



Generates Stimulus



AHB Master BFM



AHB-APB Bridge DUT



APB Peripheral



Waveform Validation

Purpose

- Verifies the complete functionality of the AHB-APB Bridge.
- Generates AHB transactions using the AHB Master BFM.
- Connects the DUT (AHB-APB Bridge) with a dummy APB Peripheral.
- Observes protocol timing and signal behavior through waveform analysis.

Implemented Test Cases

Test Case

Single Write

Single Read

INCR4 Write

INCR4 Read

WRAP4 Write

WRAP4 Read

Description

Word write transaction

Word read transaction

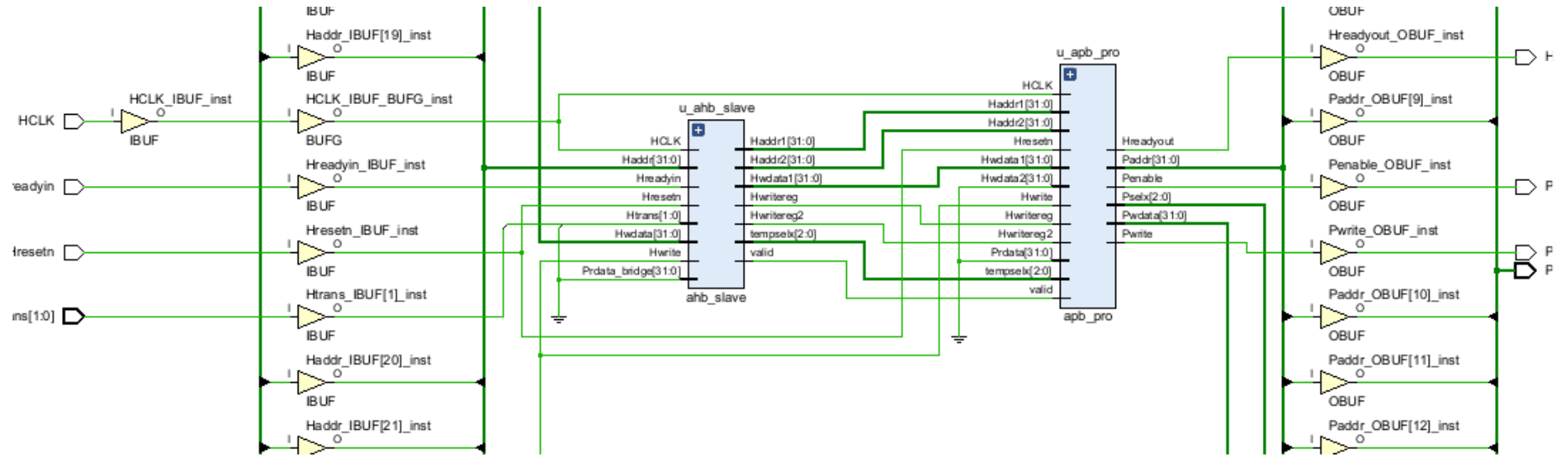
Incrementing burst write

Incrementing burst read

Wrapping burst write

Wrapping burst read

Synthesis Results – Synthesized Netlist



Synthesis Summary

- RTL successfully synthesized using **Xilinx Vivado**.
- Module hierarchy preserved after synthesis.
- Internal communication between the **AHB Slave Interface** and **APB Controller** maintained.
- Netlist generated successfully for implementation.

Conclusion

Project Summary

- Successfully designed and implemented an **AHB–APB Bridge** using Verilog HDL.
- Developed separate modules for the **AHB Master BFM**, **AHB Slave Interface**, and **APB Controller**.
- Implemented an FSM-based APB Controller to generate APB Setup and Enable phases.
- Supported **Single**, **INCR**, and **WRAP** burst transactions with configurable transfer sizes.
- Successfully verified the bridge functionality through simulation using a custom Verilog testbench.
- RTL synthesis completed successfully in **Xilinx Vivado**, validating the hardware implementation.

Key Achievements

- ✓ AHB to APB protocol conversion implemented successfully.
- ✓ Address decoding and pipelining achieved.
- ✓ Read and write transactions verified.
- ✓ Burst transfer support implemented.
- ✓ Synthesizable RTL generated successfully.